



Ministério da Educação
UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
Campus Guarapuava



Curso de Tecnologia em Sistemas para Internet
Disciplina de Fundamentos de Sistemas Computacionais
Professora Sediane Carmem Lunardi Hernandes

SISTEMA OPERACIONAL LINUX - PROCESSOS

1.1 Comandos básicos do Linux

L) Controle de Processos

Um processo é uma abstração que representa uma instância de um programa em execução. O processo pode representar:

- A execução de tarefas do próprio sistema operacional (*daemons* – processos que executam em segundo plano).
- A execução de tarefas do usuário.

Durante seu período de vida, um processo utiliza recursos do sistema, como:

- Processador para sua execução;
- Espaço em memória para armazenamento de dados e instruções;
- Emprego, direta ou indiretamente, de dispositivos de entrada e saída.

No Linux, existem vários tipos de processos, dentre eles:

- interativos: o processo precisa de entrada do usuário, ou seja, de interação. O comando *fdisk* é um processo interativo.
- *batch*: o processo não interagem com o usuário (*script* executando vários comandos de uma vez só). Por exemplo, um *script* de backup o qual salva arquivos, compacta e envia o *backup* para um servidor remoto.
- tempo real: tem prioridade sobre os processos interativos e *batch*. Utilizado em aplicações críticas onde o tempo é crucial. Por exemplo, em um drone o processo que faz o controle dos rotores é um processo em tempo real que precisa ter prioridade sobre os outros, caso contrário o drone pode cair.

Quanto as prioridades dos processos, elas podem ficar na faixa de -20 até 19, ou seja, 40 valores possíveis. Em Linux, um processo com um valor mais baixo de prioridade possui maior prioridade (+prioridade). Com isso, o processo deve ser executado antes de outros com prioridade mais baixa. Quanto maior a prioridade maior será a janela de tempo para execução. Por fim, um processo com maior prioridade acaba “ganhando” mais vezes o processador.

Em relação ao uso do processador, os processos podem ser do tipo *CPU-bound* ou *I/O-bound*. Além disso, para gerenciar os recursos de forma justa, o Linux precisa saber sobre o estado de cada um dos processos. Lembrando que os processos podem se encontrar em cada um dos seguintes estados: espera (bloqueado), pronto, executando e suspenso. O estado suspenso é um subestado do estado de espera.

COMANDOS DE CONTROLE DE PROCESSOS

O comando top - este comando exibe em tempo real informações sobre seu sistema Linux, processos em andamento e recursos do sistema, incluídos CPU, memória RAM e uso do swap, além do número total de tarefas sendo executadas

Sintaxe: top

Exercício A: Abra o firefox, um editor de texto e vários outros programas e depois execute o comando *top*. Veja como as trocas entre processos são muito dinâmicas. Agora, responda em detalhes: O que o cabeçalho do comando mostra e o que cada coluna do comando significa? Para cancelar o comando utilize a combinação de teclas CTRL + C. Lembrando que:

CTRL + C → encerra o processo em execução. CTRL + Z → suspende um processo.

Cabeçalho

- Hora atual e tempo ligado.
- Usuários conectados.
- Quantidade de processos.

Colunas

- PID: identificador do processo.
- USER: usuário dono do processo.
- PR: prioridade.
- NI: valor nice (ajuste de prioridade).
- VIRT: memória virtual total.
- RES: memória RAM utilizada.
- SHR: memória compartilhada.

Exercício B: Crie dois novos usuários pedro e madalena. Depois, alterne para o usuário pedro e digite o comando top no terminal dele. Novamente, alterne para o usuário madalena e digite top no terminal dele. Por fim, alterne para o seu usuário e digite top no seu terminal. Perceba as atividades em tempo real dos diferentes usuários do SO Linux e seus processos. Faça um *print* de tela salvando-o. Você pode fazer isso alternando o usuário utilizando a parte gráfica do Sistema Operacional.

```

top - 19:31:45 up 41 min, 6 users, load average: 2,14, 1,95, 1,52
Tarefas: 581 total, 2 em exec., 579 dormindo, 0 parado, 0 zumbi
%CPU(s): 6,3 us, 4,0 sy, 0,0 ni, 89,5 id, 0,2 wa, 0,0 hi, 0,0 si, 0,0 st
MB mem : 15680,3 total, 367,9 free, 9028,4 used, 8791,9 buff/cache
MB swap: 4096,0 total, 4096,0 free, 0,0 used, 6651,9 avail mem

  PID USUARIO   PR  NI   VIRT   RES   SHR  S  %CPU  %MEM   TEMPO+ COMANDO
-----
 7547 utfpr     20   0 1392,8g 623980 560964 S  64,1  3,9 12:22.75 code
 6970 utfpr     20   0 1394,4g 246620 166848 S  20,6  1,5  4:10.10 code
 7105 utfpr     20   0  32,5g 151188 134292 S  11,5  0,9  2:44.82 code
14717 madalena  20   0 5044336 322208 132852 R  10,7  2,0  0:02.86 gnome-shell
 3739 utfpr     20   0  875136 123248 90820 S   6,9  0,8  1:39.78 Xwayland
 9477 utfpr     20   0 3566960 152964 95320 S   2,3  1,0  0:01.02 Privileged Cont
15866 madalena  20   0  562060  65192 51872 S   2,3  0,4  0:00.38 gnome-terminal-
  85 root       20   0      0      0   0 I   0,8  0,0  0:04.84 kworker/11:0-i915-unordered
 106 root       20   0      0      0   0 I   0,8  0,0  0:00.41 kworker/2:1-mm_percpu_wq
 117 root       20   0      0      0   0 I   0,8  0,0  0:00.53 kworker/6:1-i915-unordered
1220 root       20   0  311268  8752  7660 S   0,8  0,1  0:00.32 accounts-daemon
2264 utfpr     20   0 5313736 314936 146312 S   0,8  2,0  1:07.15 gnome-shell
2954 utfpr     20   0 1821352 147596 97844 S   0,8  0,9  0:01.67 xdg-desktop-por
3733 utfpr     20   0  49,1g 205216 126676 S   0,8  1,3  0:26.21 chrome
7187 utfpr     20   0 1393,6g 151976 80336 S   0,8  0,9  0:17.29 code
7518 utfpr     20   0 1393,9g 389088 125344 S   0,8  2,4  1:24.38 code
9252 utfpr     20   0 4435480 534980 240244 S   0,8  3,3  0:08.77 firefox
15270 madalena  20   0 3872744 205956 119624 S   0,8  1,3  0:01.20 gnome-initial-s
15725 madalena  20   0 418496 29180 22284 S   0,8  0,2  0:00.09 xdg-desktop-por

```

```

top - 19:30:53 up 40 min, 4 users, load average: 1,95, 1,87, 1,47
Tarefas: 477 total, 1 em exec., 476 dormindo, 0 parado, 0 zumbi
%CPU(s): 6,8 us, 4,7 sy, 0,0 ni, 85,8 id, 2,5 wa, 0,0 hi, 0,2 si, 0,0 st
MB mem : 15680,3 total, 1127,0 free, 8113,2 used, 8649,7 buff/cache
MB swap: 4096,0 total, 4096,0 free, 0,0 used, 7567,1 avail mem

  PID USUARIO   PR  NI   VIRT   RES   SHR  S  %CPU  %MEM   TEMPO+ COMANDO
-----
 7547 utfpr     20   0 1392,8g 619200 556280 S  74,2  3,9 11:49.58 code
 6970 utfpr     20   0 1394,4g 247116 164144 S  22,0  1,5  3:59.23 code
 7105 utfpr     20   0  32,5g 153020 136088 S  14,4  1,0  2:38.26 code
 3739 utfpr     20   0 883392 123248 90820 S   9,1  0,8  1:35.61 Xwayland
11581 pedro     20   0 5084744 319300 133480 S   9,1  2,0  0:03.33 gnome-shell
12666 pedro     20   0  705984  67944 54048 S   3,8  0,4  0:00.34 gnome-terminal-
12686 pedro     20   0  12128  6276  3936 R   1,5  0,0  0:00.03 top
 114 root       20   0      0      0   0 I   0,8  0,0  0:00.41 kworker/5:1-i915-unordered
 145 root       0 -20      0      0   0 I   0,8  0,0  0:00.18 kworker/0:1H-i915_cleanup
7187 utfpr     20   0 1393,6g 153400 80336 S   0,8  1,0  0:16.99 code
7536 utfpr     20   0 1392,2g 136832 103164 S   0,8  0,9  0:01.05 code
 8664 root       20   0      0      0   0 I   0,8  0,0  0:00.94 kworker/u48:2-i915
10887 utfpr     20   0 1392,4g 391976 139528 S   0,8  2,4  0:28.43 chrome
12160 pedro     20   0 3217704 71440 55488 S   0,8  0,4  0:00.35 gjs
  1 root       20   0  23820  15476 9876 S   0,0  0,1  0:03.03 systemd
  2 root       20   0      0      0   0 S   0,0  0,0  0:00.00 kthreadd
  3 root       20   0      0      0   0 S   0,0  0,0  0:00.00 pool_workqueue_release
  4 root       0 -20      0      0   0 I   0,0  0,0  0:00.00 kworker/R-rcu_gp
top - 19:30:56 up 40 min, 4 users, load average: 1,95, 1,87, 1,47
Tarefas: 478 total, 1 em exec., 477 dormindo, 0 parado, 0 zumbi
%CPU(s): 7,1 us, 4,7 sy, 0,0 ni, 85,8 id, 2,5 wa, 0,0 hi, 0,2 si, 0,0 st
MB mem : 15680,3 total, 1127,0 free, 8113,2 used, 8649,7 buff/cache
MB swap: 4096,0 total, 4096,0 free, 0,0 used, 7567,1 avail mem

```

O comando **ps** – este comando mostra uma lista de processos, ou seja, apresenta um quadro atual dos processos, porém estático daqueles que estão sendo executados no sistema.

Sintaxe: ps [-aélux]

Opções	Significado
-e	Apresenta a linha de comando completa
-l	Gera a saída em formato longo

-a	Inclui informações sobre processos pertencentes a todos os usuários
-u	Produz saída orientada a usuário
-x	Inclui processos não associados a terminais

Exercício C: Teste todas as opções do comando `ps` e também `ps -aux`, `ps -u <seu_nome_usuario>`. Agora anote o que o comando `ps` faz, especialmente o que acontece quando você testa `ps -aux` e `ps -u <seu_nome_usuario>`?

- `ps`: mostra apenas os processos associados ao terminal atual.
- `ps aux`: exibe todos os processos do sistema, de todos os usuários, incluindo uso de CPU, memória e o comando executado.
- `ps -u <seu_nome_usuario>`: mostra somente os processos pertencentes ao usuário informado.

Exercício D: No terminal do Linux digite `man ps`. Agora veja que aparecem várias opções de uso do comando. Em especial teste os comandos abaixo e explique a saída deles.

```
$ ps aux
$ ps axjf
$ ps -U root -u root u
$ ps -ao pid,user,cmd
```

- `ps aux`: mostra todos os processos do sistema com informações detalhadas
- `ps axjf`: exibe todos os processos em formato de árvore, mostrando a relação entre processos pais e filhos.
- `ps -U root -u root u`: lista apenas os processos do usuário root em formato detalhado.
- `ps -ao pid,user,cmd`: mostra todos os processos exibindo apenas PID, usuário e comando.

O comando `pidof` – este comando é utilizado para descobrir o PID do processo (*Process IDentifier*, ou identificador de processo).

Sintaxe: `pidof <nome_do_processo>`

Exercício E: Abra o firefox, um editor de texto (pico &) e vários outros programas (p.e., gedit) e depois execute o comando *pidof*. Verifique se o comando faz o que se propõe.

```
$ pidof mysqld
$ pidof pico
$ pidof firefox
$ pidof gedit
```

Responda: Qual o retorno de cada um dos comandos acima?

```
utfpr@GP-LAB4-W1003:~$ pidof mysqld
utfpr@GP-LAB4-W1003:~$ pidof pico
utfpr@GP-LAB4-W1003:~$ pidof firefox
5620 5410
utfpr@GP-LAB4-W1003:~$ pidof gedit
utfpr@GP-LAB4-W1003:~$
```

O comando Kill – este comando permite enviar um sinal para um processo.

Sintaxe: kill [-sinal] pid...

kill -l (mostra os nomes de sinais aceitos pelo comando)

Onde,

Sinal é um evento gerado pelo SO ou por outros processos.

Exemplo:

```
$ps
2047 ... ps
2142 ... firefox
2288 ... bash
$ kill -9 2142 (kill -9 → mata o processo)
$ firefox & (pid 2335) --> se não sabe pid pede: $pidof firefox
$ ps
Para parar processo: $kill -STOP 2335
Para continuar: $kill -CONT 2335
```

Exercício F: Execute os comandos acima, anote o que acontece e tire um *print* de tela salvando-o. Depois, coloque em execução pelo menos 2 programas (processos). Por exemplo:

\$ firefox &

\$ pico &

Pare o firefox usando kill -STOP <pid> e tente navegar na web. O que acontece? Agora faça o firefox voltar a executar com kill -CONT <pid>.

Execute o comando Kill e mate os dois processos que você colocou em execução.

O comando reiniciou o usuario

```
utfpr@GP-LAB4-W1003:~$ pidof firefox
11606 11407
utfpr@GP-LAB4-W1003:~$ kill -9 11606
utfpr@GP-LAB4-W1003:~$ kill -9 11407
utfpr@GP-LAB4-W1003:~$
```

encerrou o firefox

```
utfpr@GP-LAB4-W1003:~$ firefox &
[1] 12801
utfpr@GP-LAB4-W1003:~$ Gtk-Message: 18:53:49.885: Not loading module "atk-bridge": The functionality is provided by GTK natively. Please try to not load it.
utfpr@GP-LAB4-W1003:~$
```

Execução em *Background* (segundo plano)

Libera a linha de comando para o usuário imediatamente depois de colocar o **&** (e comercial) depois do nome do programa chamado. Desta forma, o processo executará em segundo plano.

Exemplos:

```
$ firefox &
$ mcopy resposta.txt resposta2.txt &
$ ls *.txt
...
$ gedit      (aqui o gedit executa em foreground, ou seja, em primeiro plano prendendo o terminal)
$ ^Z        (suspende o processo que está em foreground chamado gedit)
```

Exercício G: Execute os comandos acima, anote o que acontece e tire um *print* de tela salvando-o.

```
utfpr@GP-LAB4-W1003:~$ mcopy resposta.txt resposta2.txt &
[1] 13350
utfpr@GP-LAB4-W1003:~$ resposta.txt: Arquivo ou diretório inexistente
a
a: comando não encontrado
[1]+  Fim da execução com status 1      mcopy resposta.txt resposta2.tx
utfpr@GP-LAB4-W1003:~$ ls *txt
cidades.txt  estados.txt  telefones.txt
utfpr@GP-LAB4-W1003:~$ mcopy telefones.txt telefones2.txt &
[1] 13395
utfpr@GP-LAB4-W1003:~$ ls *.txt
cidades.txt  estados.txt  telefones2.txt  telefones.txt
[1]+  Concluído      mcopy telefones.txt telefones2.txt
utfpr@GP-LAB4-W1003:~$ gedit
Comando 'gedit' não encontrado, mas poder ser instalado com:
sudo snap install gedit # version 48.1, or
sudo apt install gedit # version 46.1-3
Veja 'snap info gedit' para versões adicionais.
utfpr@GP-LAB4-W1003:~$ ^z
bash: :s^z: substitution failed
utfpr@GP-LAB4-W1003:~$
```

O comando jobs - este comando lista processos suspensos e em *background*.

Sintaxe: jobs

Exercício H: Quais processos estão suspensos (parados) e em *background* em seu SO? Anote e faça um print de tela salvando-o.

```
utfpr@GP-LAB4-W1003:~$ jobs
[1]+  Parado                pico n.txt
utfpr@GP-LAB4-W1003:~$
```

O comando bg – este comando coloca um processo em *background*.

Sintaxe: bg[%n]

Onde,

n é o número do job desejado

Para descobrir o número do job desejado você digita *jobs* e então ele aparecerá entre colchetes.

Outra forma de colocar processo em *background* é

1. Primeiro: parar o processo usando CTRL +Z.
2. Segundo: digitar o comando bg.

```
utfpr@GP-LAB4-W1003:~$ bg
[1]+ pico n.txt &
utfpr@GP-LAB4-W1003:~$
```

Exemplo:

```
$ gedit (entre no gedit e digite algo)
$ CTRL + Z (tente digitar depois desse comando, o que acontece?)
$ jobs
[1]+ gedit
$ bg %1 (número que apareceu entre colchetes antes do nome do processo)
```

Exercício I: Faça o que se pede e ao final dê um print de tela mostrando o comportamento dos comandos digitados.

Inicie dois processos (*firefox* e *gedit*)

```
$ firefox
$ CTRL + Z (suspende processo)
$ gedit
$ CTRL + Z (suspende processo)
```

Coloque-os em *background*

```
$ jobs
$ bg %? → (número que aparece entre colchetes antes do nome do processo firefox)
$ bg %? → (número que aparece entre colchetes antes do nome do processo gedit)
```

Pare o processo *gedit*, mas depois faça-o voltar a executar.

```
$ pidof gedit
$ kill -STOP pid_de_gedit
```

Tente escrever no arquivo aberto no *gedit*.

Conseguiu? () Sim (X) Não

Agora faça o processo continuar a executar

```
$ kill -CONT pid_de_gedit
```

Mate os processos *firefox* e *gedit* utilizando o comando *kill -9*.

Repita os exercícios novamente criando vários processos, parando-os e depois fazendo-os voltar a executar. Use sua criatividade, dê um print de tela salvando-o.

```
utfpr@GP-LAB4-W1003:~$ firefox
Gtk-Messsage: 19:10:28.778: Not loading module "atk-bridge": The functionality is provided by GTK natively. Please try to not load it.
utfpr@GP-LAB4-W1003:~$ gedit
^Z
[2]+  Parado                  gedit
utfpr@GP-LAB4-W1003:~$ jobs
[1]-  Parado                  pico n.txt
[2]+  Morto                   gedit
utfpr@GP-LAB4-W1003:~$ bg %2
bash: bg: %2: trabalho não existe
utfpr@GP-LAB4-W1003:~$ gedit
^Z
[2]+  Parado                  gedit
utfpr@GP-LAB4-W1003:~$ jobs
[1]-  Parado                  pico n.txt
[2]+  Parado                  gedit
utfpr@GP-LAB4-W1003:~$ bg %2
[2]+ gedit &
utfpr@GP-LAB4-W1003:~$ bg %1
[1]+ pico n.txt &
utfpr@GP-LAB4-W1003:~$ pidof gedit
15649

[1]+  Parado                  pico n.txt
utfpr@GP-LAB4-W1003:~$ kill -STOP 15649
utfpr@GP-LAB4-W1003:~$ kill -CONT 15649
utfpr@GP-LAB4-W1003:~$ kill -9 15649
bash: kill: (15649) - Processo inexistente
[2]-  Concluído              gedit
utfpr@GP-LAB4-W1003:~$
```

O comando killall – mata processos pelo nome.

Sintaxe: killall opções <nome_processo>

Opções:

-e nome : procura pelo nome que seja igual (exato) ao especificado. Se o nome tiver mais de 15 caracteres, apenas os primeiros 15 caracteres são comparados.

- i** : pede confirmação antes de matar o processo.
- l** : lista os sinais que podem ser enviados a um processo junto com o comando killall.
- s** sinal ou **-sinal** : define outro sinal a ser enviado aos processos ao invés do sinal TERM.
- u** usuário : mata apenas os processos de determinado usuário.
- v** : informa se o sinal foi enviado com sucesso.

Exemplos

```
$ killall -9 firefox
$ firefox &
$ killall -e firefox
```

O primeiro comando mata todos os processos relacionados ao firefox. O uso do sinal 9 (SIGKILL) informa que o comando não pode ser ignorado pelo aplicativo, enquanto o segundo remove apenas o processo firefox.

O comando fg – trazendo processos para *foreground* – primeiro plano.

Sintaxe: fg[%n]

Onde,

n é o número do job desejado

Para colocar o processo em *foreground* é só usar o comando *fg*.

Exemplo:

```
$ gedit
$ CTRL + Z
$ jobs
[2] gedit
$ bg %2
$ fg %2
```

Exercício J: Faça o mesmo exercício com o *firefox* e dê um *print* de tela salvando-o.

```
utfpr@GP-LAB4-W1003:~$ killall -9 firefox
utfpr@GP-LAB4-W1003:~$ firefox &
[2] 15848
utfpr@GP-LAB4-W1003:~$ Gtk-Message: 19:14:37.718: Not loading module "atk-bridge": The functionality is provided by GTK natively. Please try to not load it.

utfpr@GP-LAB4-W1003:~$ killall -e firefox
utfpr@GP-LAB4-W1003:~$ Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.
[2]- Terminado          firefox
```

```

utfpr@GP-LAB4-W1003:~$ jobs
[1]  Parado                pico n.txt
[2]-  Parado                firefox
[3]+  Parado                gedit
utfpr@GP-LAB4-W1003:~$ bg % 2
[3]+  gedit &
[2]+  firefox &
utfpr@GP-LAB4-W1003:~$ bg %2
bash: bg: o trabalho 2 já está em plano de fundo
utfpr@GP-LAB4-W1003:~$ fg %2
firefox

^Z
[2]+  Parado                firefox
utfpr@GP-LAB4-W1003:~$ █

```

Pipes (|): O pipe faz com que o canal de saída de um processo seja associado ao canal de entrada de outro processo.

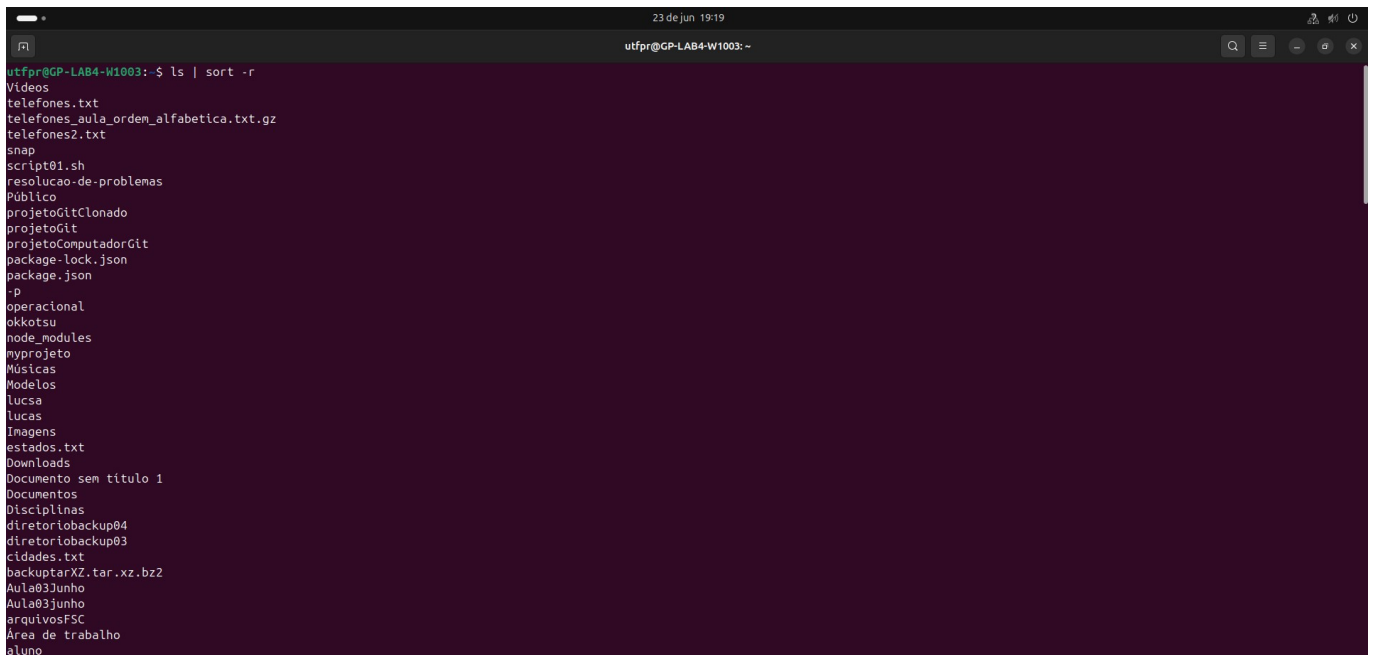
Exemplos:

```

$ls | sort -r
$ls /usr/local | more
$ps -aux | more

```

Exercício K Teste os comandos dos exemplos acima e faça um *print* de tela.



```

utfpr@GP-LAB4-W1003:~$ ls | sort -r
Videos
telefones.txt
telefones_aula_orden_alfabetica.txt.gz
telefones2.txt
snap
script01.sh
resolucao-de-problemas
Público
projetoGitClonado
projetoGit
projetoComputadorGit
package-lock.json
package.json
-p
operacional
okkotsu
node_modules
myprojeto
Músicas
Modelos
lucsa
lucas
Imagens
estados.txt
Downloads
Documento sem titulo 1
Documentos
Disciplinas
diretoriobackup04
diretoriobackup03
cidades.txt
backuptarX2.tar.xz.bz2
Aula03Junho
Aula03junho
arquivosFSC
Area de trabalho
aluno

```

```

utfpr@GP-LAB4-W1003:~$ ls /home/utfpr | more
aluno
Área de trabalho
arquivosFSC
Aula03junho
Aula03Junho
backuptarXZ.tar.xz.bz2
cidades.txt
diretoriobackup03
diretoriobackup04
Disciplinas
Documentos
Documento sem titulo 1
Downloads
Estados.txt
Imagens
lucas
lucsa
Modelos
Músicas
myprojeto
node_modules
okkotsu
operacional
ip
package.json
package-lock.json
projetoComputadorGit
projetoGit
projetoGitClonado
Público
resolucao-de-problemas
script01.sh
snap
telefones2.txt
telefones_aula_orden_alfabetica.txt.gz
telefones.txt
Videos

```

```

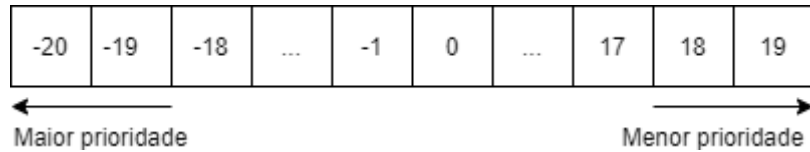
23 de jun 19:19
utfpr@GP-LAB4-W1003:~$ ps -aux | more
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.1  0.0  24032 15684 ?        Ss   18:41   0:03 /sbin/init sp
root        22  0.0  0.0      0     0 ?        Ss   18:41   0:00 [kthreadd]
root        33  0.0  0.0      0     0 ?        Ss   18:41   0:00 [pool_workque
root        44  0.0  0.0      0     0 ?        I<   18:41   0:00 [kworker/R-rc
root        55  0.0  0.0      0     0 ?        I<   18:41   0:00 [kworker/R-sy
root        66  0.0  0.0      0     0 ?        I<   18:41   0:00 [kworker/R-kv
root        77  0.0  0.0      0     0 ?        I<   18:41   0:00 [kworker/R-sl
root        88  0.0  0.0      0     0 ?        I<   18:41   0:00 [kworker/R-ne
root        99  0.0  0.0      0     0 ?        I<   18:41   0:00 [kworker/0:0H
root       110  0.0  0.0      0     0 ?        I<   18:41   0:00 [kworker/0:1-
root       121  0.0  0.0      0     0 ?        I<   18:41   0:00 [kworker/R-mm
root       132  0.0  0.0      0     0 ?        Ss   18:41   0:00 [ksoftirqd/0]
root       143  0.1  0.0      0     0 ?        I<   18:41   0:03 [rcu_preempt]
root       154  0.0  0.0      0     0 ?        Ss   18:41   0:00 [rcu_exp_par_
root       165  0.0  0.0      0     0 ?        Ss   18:41   0:00 [rcu_exp_gp_k
root       176  0.0  0.0      0     0 ?        Ss   18:41   0:00 [migration/0]
root       187  0.0  0.0      0     0 ?        Ss   18:41   0:00 [idle_inject/
root       198  0.0  0.0      0     0 ?        Ss   18:41   0:00 [cpuhp/0]
root       209  0.0  0.0      0     0 ?        Ss   18:41   0:00 [cpuhp/2]
root       220  0.0  0.0      0     0 ?        Ss   18:41   0:00 [idle_inject/
root       231  0.0  0.0      0     0 ?        Ss   18:41   0:00 [migration/2]
root       242  0.0  0.0      0     0 ?        Ss   18:41   0:00 [ksoftirqd/2]
root       253  0.0  0.0      0     0 ?        I<   18:41   0:00 [kworker/2:0H
root       264  0.0  0.0      0     0 ?        Ss   18:41   0:00 [cpuhp/4]
root       275  0.0  0.0      0     0 ?        Ss   18:41   0:00 [idle_inject/
root       286  0.0  0.0      0     0 ?        Ss   18:41   0:00 [migration/4]
root       297  0.0  0.0      0     0 ?        Ss   18:41   0:00 [ksoftirqd/4]
root       308  0.0  0.0      0     0 ?        I<   18:41   0:00 [kworker/4:0H

```

O comando nice e renice – permite influenciar no cálculo da prioridade dos processos que estão executando no Linux. Cada processo possui um valor de prioridade que influencia em seu escalonamento. A prioridade de execução de um processo normal pode variar de 0 (maior prioridade) a 39 (menor prioridade). Nesse caso a prioridade é calculada como

$$\text{PRI} = 20 + \text{NI}$$

Onde, NI é o valor *nice* utilizado para alterar a prioridade de um comando do usuário. O NI pode variar de -20 a 19. Relembrando que quanto menor for o número maior será a sua prioridade. Observe a figura abaixo.



O *nice* determina a prioridade do processo quando ele é criado e o *renice* modifica a prioridade dos processos em execução. O usuário só consegue utilizar prioridades positivas, entretanto o administrador utiliza todas elas. No Linux, os processos em tempo real possuem prioridade negativa. O usuário não pode alterar as prioridades deste tipo de processo.

O comando nice

Sintaxe: `nice -n valor` ou `nice -valor`

`nice -help` para mostrar as opções disponíveis com o comando.

`--version` para exibir informações sobre o comando.

Exercício L: Execute os comandos *nice* e *renice* abaixo tire *print* de tela salvando as execuções.

```
$ nice top
```

O comando *top* é executado com NI = 10 e PR = 30, pois quando o valor NI não é especificado com o comando *nice*, assume-se que NI = 10. Neste caso, se o comando *top* tivesse sido chamado sem o comando *nice*, o sistema teria usado NI = 0 e PR = 20.

```
$ nice -6 top ou nice -n 6 top //diminui a
```

prioridade Neste caso, o comando é executado com NI = 6 e PR =

26.

O comando

```
sudo nice --6 top
```

ou

```
sudo nice -n -6 top
```

faz com que o comando seja executado com NI = -6 e PR = 14. Isto significa que foi aumentada a prioridade de execução. Somente o administrador do sistema (root) pode definir um valor negativo para *nice*. Caso o comando não seja digitado com *sudo*, o valor da prioridade não será alterado (NI = 0, PR = 20).

Caso seja fornecido um valor NI fora dos limites, o sistema usa o valor mais próximo possível. Assim, para `nice -30 top`

temos NI = 19 e PR = 39. E para

```
sudo nice --30 top
```

```

23 de jun 19:21
utfpr@GP-LAB4-W1003: ~
utfpr@GP-LAB4-W1003:~$ nice top

top - 19:21:16 up 39 min, 2 users, load average: 0,44, 0,43, 0,49
Tarefas: 366 total, 2 em exec., 352 dormindo, 12 parado, 0 zumbi
%CPU(s): 1,3 us, 0,6 sy, 0,0 ni, 98,0 id, 0,0 wa, 0,0 hi, 0,0 si, 0,0 st
MB mem : 15680,3 total, 3456,7 free, 4511,9 used, 9091,6 buff/cache
MB swap: 4096,0 total, 4096,0 free, 0,0 used, 11168,4 avail mem

  PID USUARIO  PR  NI  VIRT  RES  SHR  S  %CPU  %MEM  TEMPO+ COMANDO
  9763 utfpr    20   0  5706608 365712 174440 S  10,9  2,3  2:23.88 gnome-shell
 13750 utfpr    20   0 1394,1g 267964 137504 S  5,0  1,7  0:54.99 chrome
 13827 utfpr    20   0 708920 71096 54400 R  2,3  0,4  0:10.76 gnome-terminal-
10549 utfpr    20   0  49,0g 215212 136832 S  1,8  1,3  0:19.79 chrome
10495 utfpr    20   0  48,9g 378100 278556 S  1,4  2,4  0:12.55 chrome
10783 utfpr    20   0 1395,7g 339964 144540 S  1,4  2,1  0:22.06 chrome
11166 utfpr    20   0 9100672 596704 207612 S  1,4  3,7  1:10.41 soffice.bin
  131 root      20   0      0      0      0  I  0,5  0,0  0:03.43 kworker/11:1-i915-unordered
  198 root      0 -20      0      0      0  I  0,5  0,0  0:00.12 kworker/1:1H-i915_cleanup
 6685 root      20   0      0      0      0  I  0,5  0,0  0:01.19 kworker/u48:0-flush-8:0
 8536 root      20   0      0      0      0  I  0,5  0,0  0:01.01 kworker/u48:2-i915
 9907 utfpr    20   0 386712 13132 7648 S  0,5  0,1  0:07.94 ibus-daemon
17194 root      0 -20      0      0      0  I  0,5  0,0  0:00.51 kworker/u49:1-i915_flip
   1 root      20   0 24032 15684 9876 S  0,0  0,1  0:04.00 systemd
   2 root      20   0      0      0      0  S  0,0  0,0  0:00.00 kthreadd
   3 root      20   0      0      0      0  S  0,0  0,0  0:00.00 pool_workqueue_release
   4 root      0 -20      0      0      0  I  0,0  0,0  0:00.00 kworker/R-rcu_gp

```

```

23 de jun 19:22
utfpr@GP-LAB4-W1003: ~
utfpr@GP-LAB4-W1003:~$ top

top - 19:21:58 up 40 min, 2 users, load average: 0,43, 0,43, 0,48
Tarefas: 368 total, 1 em exec., 353 dormindo, 14 parado, 0 zumbi
%CPU(s): 1,1 us, 0,3 sy, 0,0 ni, 98,4 id, 0,1 wa, 0,0 hi, 0,0 si, 0,0 st
MB mem : 15680,3 total, 3436,2 free, 4531,9 used, 9094,1 buff/cache
MB swap: 4096,0 total, 4096,0 free, 0,0 used, 11148,4 avail mem

  PID USUARIO  PR  NI  VIRT  RES  SHR  S  %CPU  %MEM  TEMPO+ COMANDO
  9763 utfpr    20   0 5716636 381712 182300 S  5,6  2,4  2:27.15 gnome-shell
 13750 utfpr    20   0 1394,1g 268260 137504 S  5,6  1,7  0:57.44 chrome
 13827 utfpr    20   0 709784 72024 54400 S  5,0  0,4  0:12.26 gnome-terminal-
10549 utfpr    20   0  49,0g 215212 136832 S  1,7  1,3  0:20.47 chrome
  106 root      20   0      0      0      0  I  0,3  0,0  0:00.42 kworker/2:1-mm_percpu_wq
 1953 root      20   0 982424 82260 29244 S  0,3  0,5  0:03.18 mono
 6685 root      20   0      0      0      0  I  0,3  0,0  0:01.22 kworker/u48:0-events_unbound
17194 root      0 -20      0      0      0  I  0,3  0,0  0:00.66 kworker/u49:1-i915_flip
17625 utfpr    26   6 12144 6256 3968 R  0,3  0,0  0:00.02 top
   1 root      20   0 24032 15684 9876 S  0,0  0,1  0:04.00 systemd
   2 root      20   0      0      0      0  S  0,0  0,0  0:00.00 kthreadd
   3 root      20   0      0      0      0  S  0,0  0,0  0:00.00 pool_workqueue_release
   4 root      0 -20      0      0      0  I  0,0  0,0  0:00.00 kworker/R-rcu_gp
   5 root      0 -20      0      0      0  I  0,0  0,0  0:00.00 kworker/R-sync_wq
   6 root      0 -20      0      0      0  I  0,0  0,0  0:00.00 kworker/R-kvfree_rcu_reclaim
   7 root      0 -20      0      0      0  I  0,0  0,0  0:00.00 kworker/R-slub_flushwq
   8 root      0 -20      0      0      0  I  0,0  0,0  0:00.00 kworker/R-netns
  10 root      0 -20      0      0      0  I  0,0  0,0  0:00.00 kworker/0:0H-events_highpri
  11 root      20   0      0      0      0  I  0,0  0,0  0:00.44 kworker/0:1-events
  13 root      0 -20      0      0      0  I  0,0  0,0  0:00.00 kworker/R-mm_percpu_wq
  14 root      20   0      0      0      0  S  0,0  0,0  0:00.11 ksoftirqd/0
  15 root      20   0      0      0      0  I  0,0  0,0  0:03.82 rcu_preempt
  16 root      20   0      0      0      0  S  0,0  0,0  0:00.00 rcu_exp_par_gp_kthread_worker/0
  17 root      20   0      0      0      0  S  0,0  0,0  0:00.01 rcu_exp_gp_kthread_worker
  18 root      rt   0      0      0      0  S  0,0  0,0  0:00.01 migration/0
  19 root      -51  0      0      0      0  S  0,0  0,0  0:00.00 idle_inject/0
  20 root      20   0      0      0      0  S  0,0  0,0  0:00.00 cpuhp/0
  21 root      20   0      0      0      0  S  0,0  0,0  0:00.00 cpuhp/2
  22 root      -51  0      0      0      0  S  0,0  0,0  0:00.00 idle_inject/2
  23 root      rt   0      0      0      0  S  0,0  0,0  0:00.07 migration/2
  24 root      20   0      0      0      0  S  0,0  0,0  0:00.06 ksoftirqd/2
  26 root      0 -20      0      0      0  I  0,0  0,0  0:00.00 kworker/2:0H-events_highpri
  27 root      20   0      0      0      0  S  0,0  0,0  0:00.00 cpuhp/4
  28 root      -51  0      0      0      0  S  0,0  0,0  0:00.00 idle_inject/4
  29 root      rt   0      0      0      0  S  0,0  0,0  0:00.07 migration/4
  30 root      20   0      0      0      0  S  0,0  0,0  0:00.09 ksoftirqd/4
  32 root      0 -20      0      0      0  I  0,0  0,0  0:00.00 kworker/4:0H-events_highpri
  33 root      20   0      0      0      0  S  0,0  0,0  0:00.00 cpuhp/6

```

```

23 de jun 19:22
utfpr@GP-LAB4-W1003: ~
top - 19:22:32 up 40 min, 2 users, load average: 0,26, 0,39, 0,47
Tarefas: 371 total, 2 em exec., 354 dormindo, 15 parado, 0 zumbi
%CPU(s): 0,8 us, 0,4 sy, 0,0 ni, 98,9 id, 0,0 wa, 0,0 hi, 0,0 si, 0,0 st
MB mem : 15680,3 total, 3445,8 free, 4522,0 used, 9090,6 buff/cache
MB swap: 4096,0 total, 4096,0 free, 0,0 used, 11158,2 avail mem

  PID USUARIO  PR  NI  VIRT  RES  SHR  S  %CPU  %MEM  TEMPO+ COMANDO
13750 utfpr    20   0 1394,1g 268672 137504 S   5,6   1,7   0:59.24 chrome
 9763 utfpr    20   0 5700720 374004 174440 S   4,9   2,3   2:30.36 gnome-shell
13827 utfpr    20   0 709976 72216 54400 R   2,2   0,4   0:12.78 gnome-terminal-
10549 utfpr    20   0 49,0g 215212 136832 S   1,9   1,3   0:21.01 chrome
  48 root       20   0      0      0      0 S   0,4   0,0   0:00.07 ksoftirqd/10
11166 utfpr    20   0 9100672 596704 207612 S   0,4   3,7   1:11.97 soffice.bin
17194 root       0 -20      0      0      0 I   0,4   0,0   0:00.78 kworker/u49:1-i915_flip
17649 root      14  -6 12124 6292 3964 R   0,4   0,0   0:00.02 top
  1 root       20   0 24032 15684 9876 S   0,0   0,1   0:04.00 systemd
  2 root       20   0      0      0      0 S   0,0   0,0   0:00.00 kthreadd
  3 root       20   0      0      0      0 S   0,0   0,0   0:00.00 pool_workqueue_release
  4 root       0 -20      0      0      0 I   0,0   0,0   0:00.00 kworker/R-rcu_gp
  5 root       0 -20      0      0      0 I   0,0   0,0   0:00.00 kworker/R-sync_wq
  6 root       0 -20      0      0      0 I   0,0   0,0   0:00.00 kworker/R-kvfree_rcu_reclaim
  7 root       0 -20      0      0      0 I   0,0   0,0   0:00.00 kworker/R-slub_flushwq
  8 root       0 -20      0      0      0 I   0,0   0,0   0:00.00 kworker/R-netns
 10 root       0 -20      0      0      0 I   0,0   0,0   0:00.00 kworker/0:0H-events_highpri
 11 root       20   0      0      0      0 I   0,0   0,0   0:00.45 kworker/0:1-mm_percpu_wq
 13 root       0 -20      0      0      0 I   0,0   0,0   0:00.00 kworker/R-mm_percpu_wq

```

O comando renice

Sintaxe: renice -n valor ou renice -valor

Exemplo:

```
PID ... CMD
2020 ... firefox
3096 ... gedit
$ renice -n 10 3096
$ renice -n 5 2020
```

O comando pstree – mostra a árvore de processos.

O comando free – mostra a quantidade de memória livre e usada no sistema (testar com as opções -b, -k, -m para mostrar em Bytes, Kbytes e MegaBytes, respectivamente).

```
utfpr@GP-LAB4-W1003:~$ reinicie -n 10 16748
reinicie: comando não encontrado
utfpr@GP-LAB4-W1003:~$ renice -n 10 16748
16748 (process ID) com prioridade antiga 0, prioridade nova 10
utfpr@GP-LAB4-W1003:~$
```

M) Para não perder o compromisso

O comando date – O comando date exibe e edita a data e a hora atuais do sistema.

Exemplo:

```
$ date
$ Tue Oct 22 12:58:03 PDT 2021
$ sudo date 1022180705 2021
$ Tue Oct 22 18:07:05 PDT 2021
```

O comando cal - exibe um simples calendário. Teste em seu terminal Linux.

```
utfpr@GP-LAB4-W1003:~$ date
ter 23 jun 2026 19:27:13 -03
utfpr@GP-LAB4-W1003:~$ sudo date
ter 23 jun 2026 19:27:19 -03
utfpr@GP-LAB4-W1003:~$ cal
      Junho 2026
do se te qu qu se sá
    1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30
```

Bibliografia

NEVES, Júlio Cezar. Programação SHELL LINUX. 8a. Ed. Rio de Janeiro: Brasport, 2010. <http://wiki.ubuntu-br.org/ComandosBasicos>
[nice | Guia Linux \(uniriotec.br\)](http://wiki.ubuntu-br.org/ComandosBasicos)